

Postmortem Técnico — Lambda ShopEasy Corp

Autor: Juan Pablo Castaño

Fecha: 25 de mayo de 2026

Cuenta AWS: 654654478122

Región: us-east-1

1. Resumen del incidente

ShopEasy Corp reportó tres síntomas en una de sus funciones Lambda:

- Fallos intermitentes
- Incremento en tiempos de ejecución
- Errores aleatorios en el procesamiento

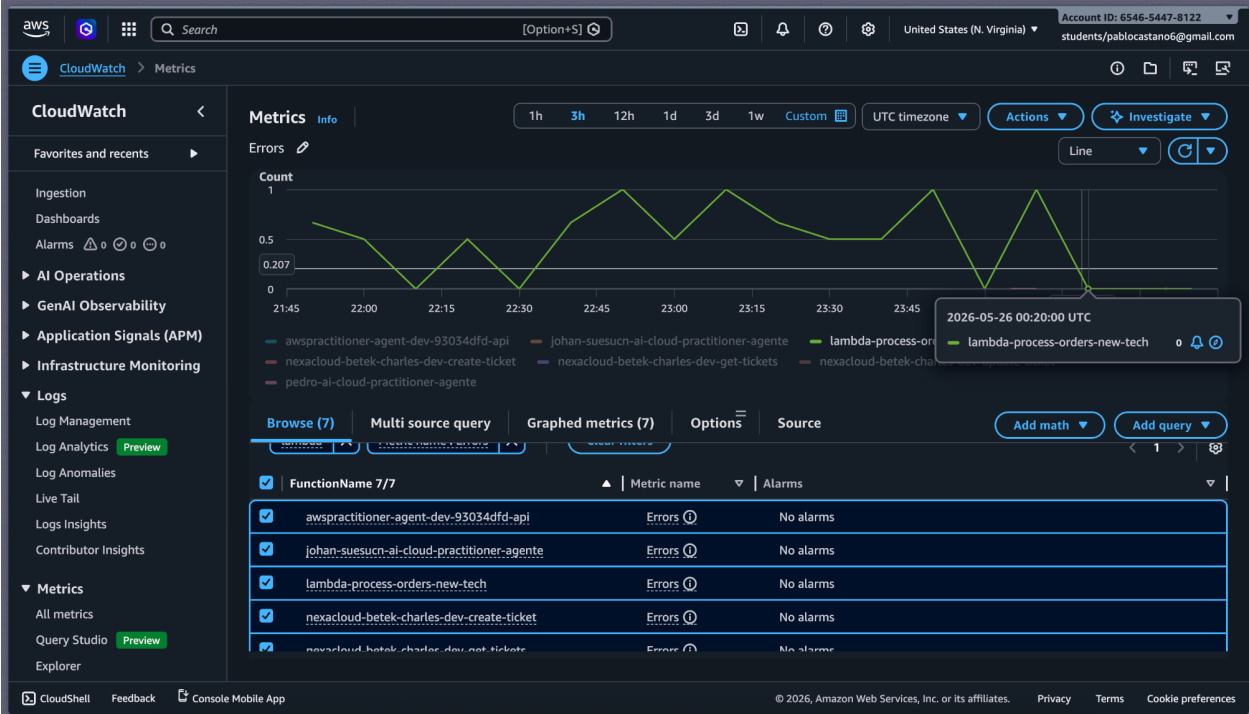
Tras el análisis con CloudWatch Logs, CloudWatch Metrics y AWS X-Ray, identifiqué que la función afectada es lambda-process-orders-new-tech. La causa raíz es una lógica aleatoria en el código que provoca errores y lentitud en aproximadamente el 75% de las invocaciones.

2. Identificación de la Lambda con fallos

Como el enunciado no indicaba qué Lambda revisar, fui a:

CloudWatch → Metrics → All metrics → Lambda → By Function Name

Seleccioné la métrica Errors para las 7 funciones de la cuenta en una ventana de 3 horas. Solo una mostró errores constantes: lambda-process-orders-new-tech. Las otras 6 funciones quedaron en cero.

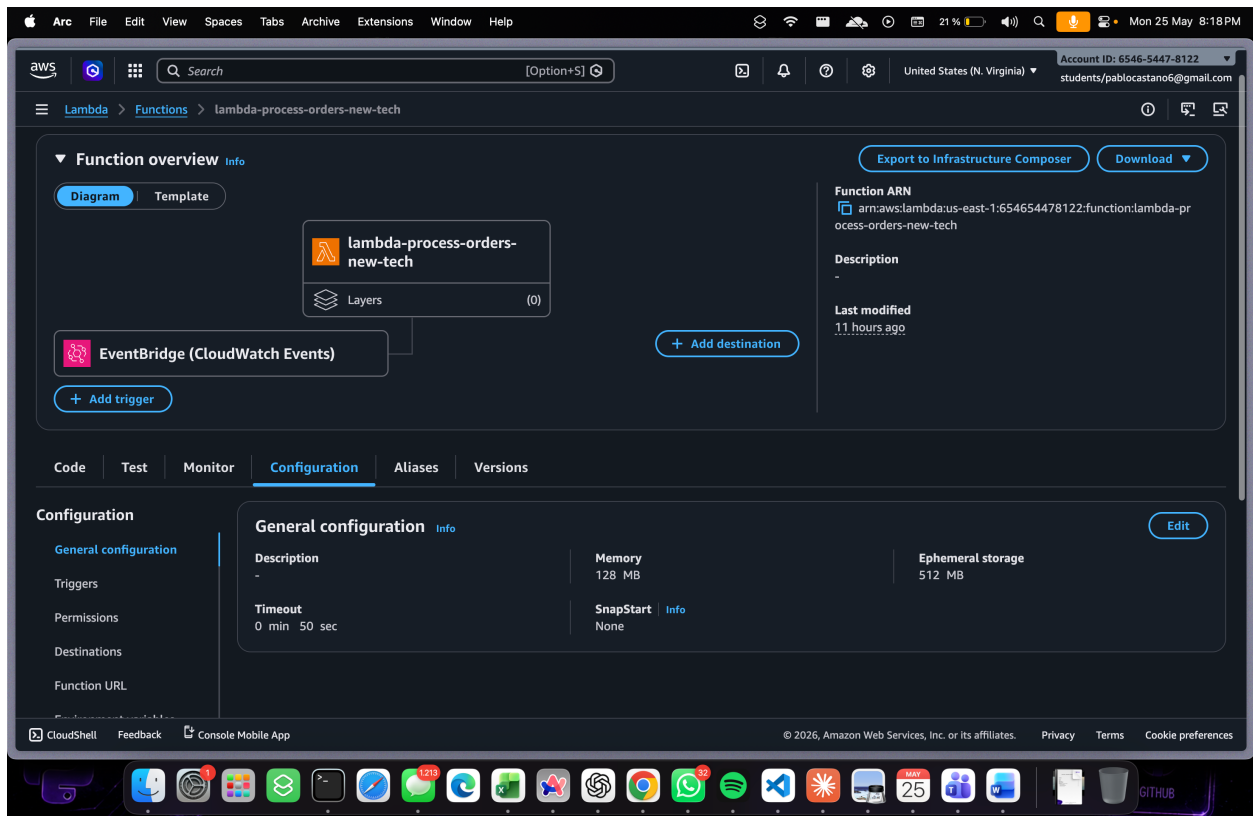


 Screenshot 1: Gráfica de CloudWatch Metrics comparando Errors de las 7 Lambdas — solo lambda-process-orders-new-tech presenta errores.

3. Datos de la función

Confirmé en la consola de Lambda los datos de la función afectada:

Atributo	Valor
Nombre	lambda-process-orders-new-tech
Runtime	Python 3.13
Memoria	128 MB
Timeout	50 segundos
Trigger	EventBridge
X-Ray Active Tracing	Habilitado
Región	us-east-1



4. Revisión del código

Al revisar el handler `lambda_function.py` se encontró el origen de los errores:

```
import json
import random
import time

def lambda_handler(event, context):

    number = random.randint(1,4)

    # Error timeout
    if number == 1:
        time.sleep(5)

    # Error aplicación
    elif number == 2:
        raise Exception("Database connection failed")

    # Error body
    elif number == 3:
        body = event["body"]
```

```

return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "Success"
    })
}

```

El código usa `random.randint(1, 4)` para elegir aleatoriamente un comportamiento:

- Si sale 1 → `time.sleep(5)`: lentitud de 5 segundos.
- Si sale 2 → lanza `Exception("Database connection failed")`.
- Si sale 3 → intenta leer `event["body"]` que no existe → `KeyError`.
- Si sale 4 → retorna éxito.

Esto explica por qué los errores son intermitentes y aleatorios: cada invocación tiene aproximadamente 75% de probabilidad de fallar o ir lenta, y solo 25% de terminar correctamente.

5. Análisis de Logs (CloudWatch Logs)

Se accedió a CloudWatch → Logs Insights, log group `/aws/lambda/lambda-process-orders-new-tech`.

5.1 Query para contar errores

```

fields @timestamp, @message
| filter @message like /ERROR/ or @message like /Exception/ or @message like /Traceback/
| stats count(*) as total by @message
| sort total desc

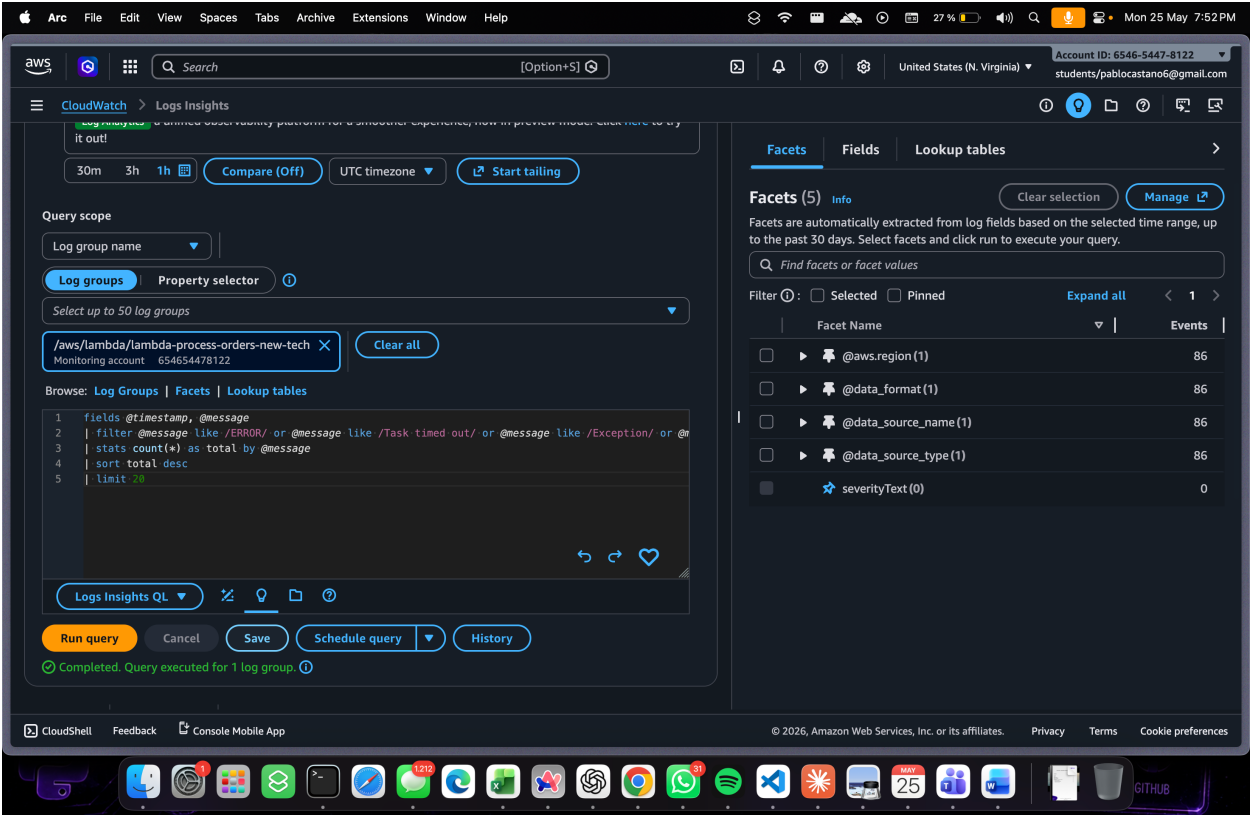
```

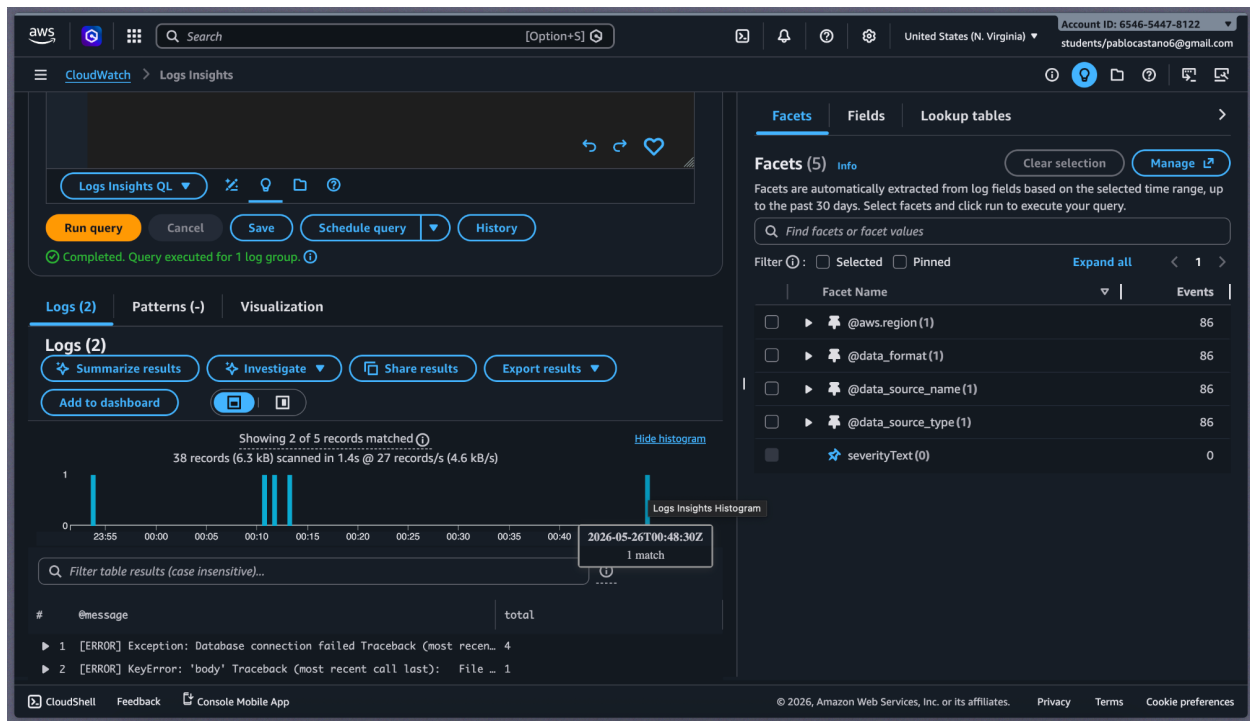
Resultados:

- [ERROR] Exception: Database connection failed → 4 veces
- [ERROR] KeyError: 'body' → 1 vez

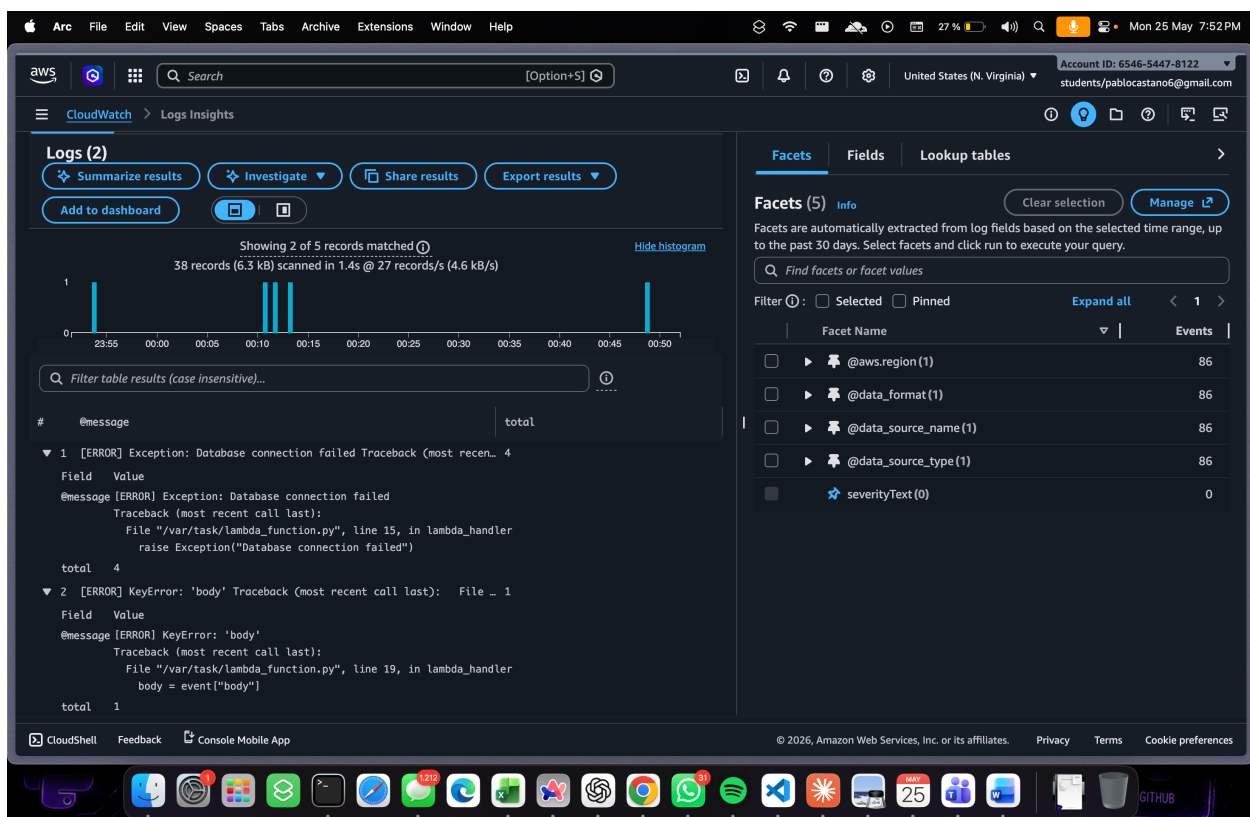
Resumen tabulado:

Error	Total	Origen
Exception: Database connection failed	4	raise Exception — línea 15
KeyError: 'body'	1	event["body"] no existe — línea 19





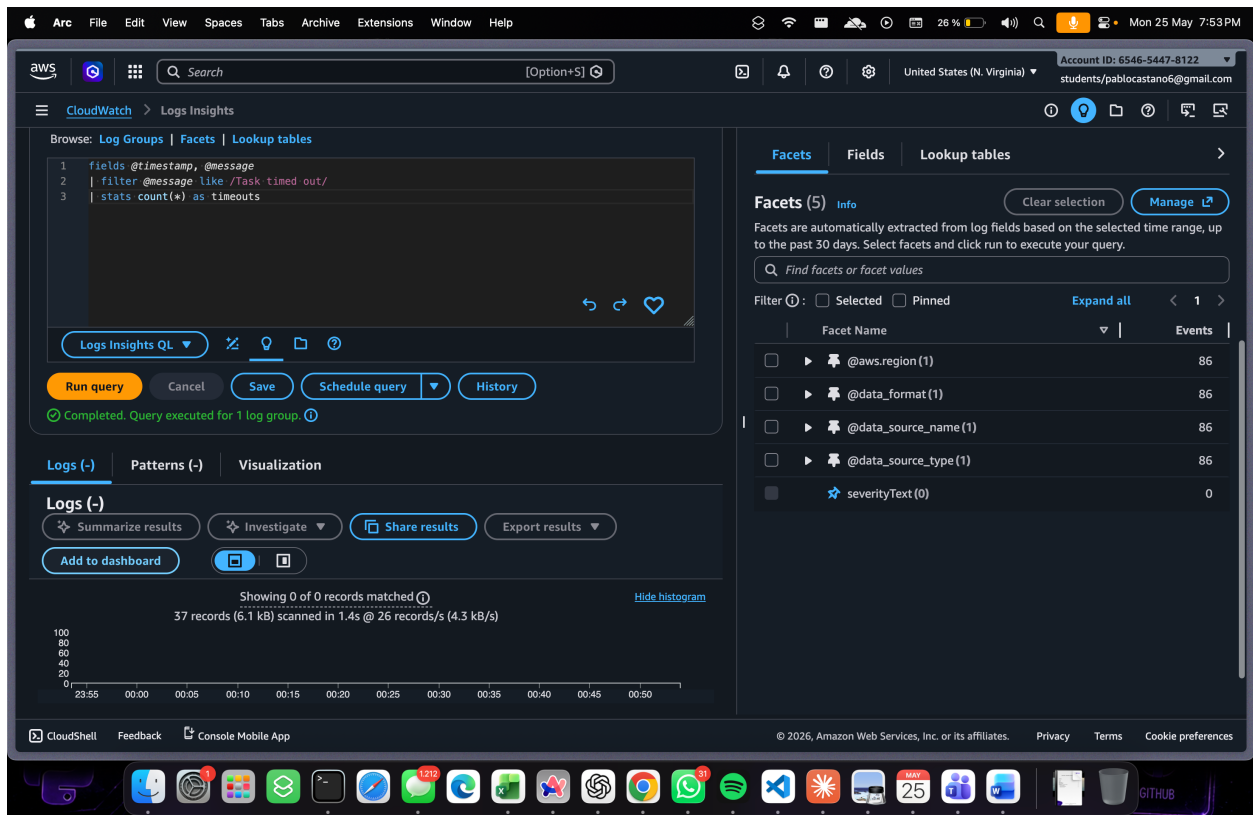
Resultado de la query de conteo de errores en Logs Insights, mostrando Database connection failed (4) y KeyError 'body' (1).



5.2 Query para verificar timeouts

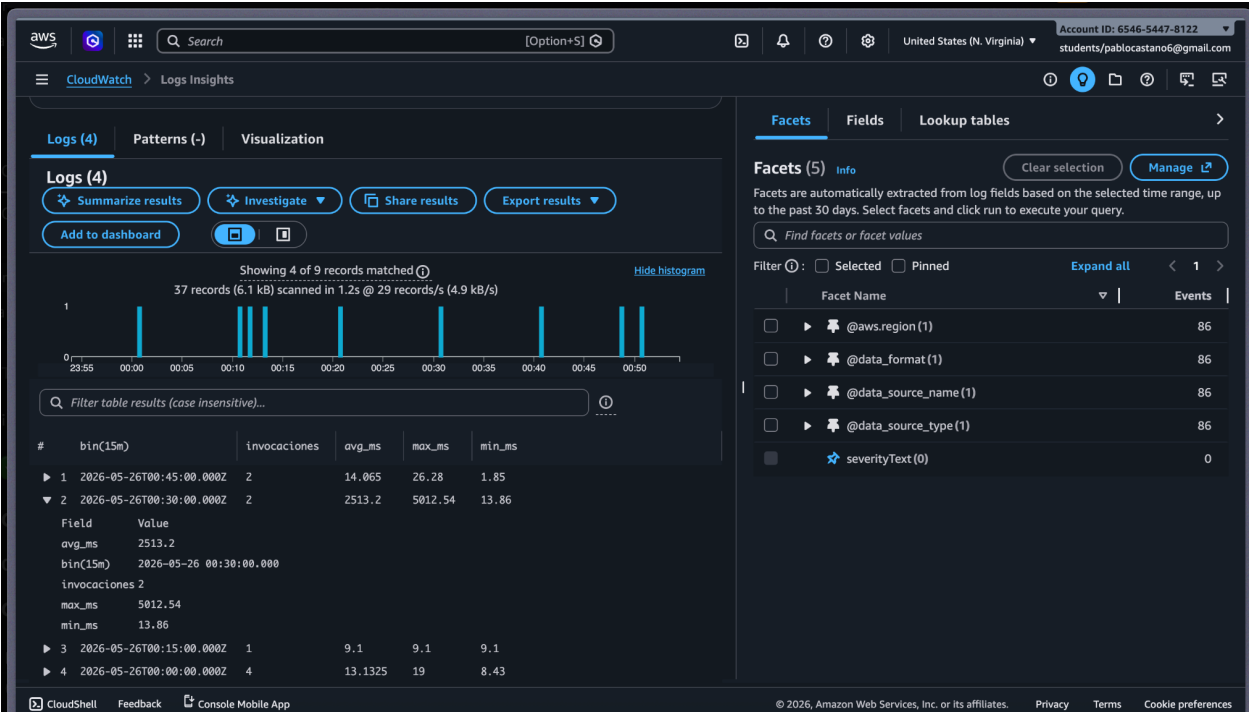
```
filter @message like /Task timed out/  
| stats count(*) as timeouts
```

Resultado: 0 timeouts. El timeout configurado (50 s) es mucho mas que sleep de 5 s, por lo que la lentitud no llega a disparar timeout.



5.3 Query de duración de invocaciones

```
filter @type = "REPORT"  
| stats count(*) as invocaciones, avg(@duration) as avg_ms, max(@duration) as max_ms,  
min(@duration) as min_ms by bin(15m)
```



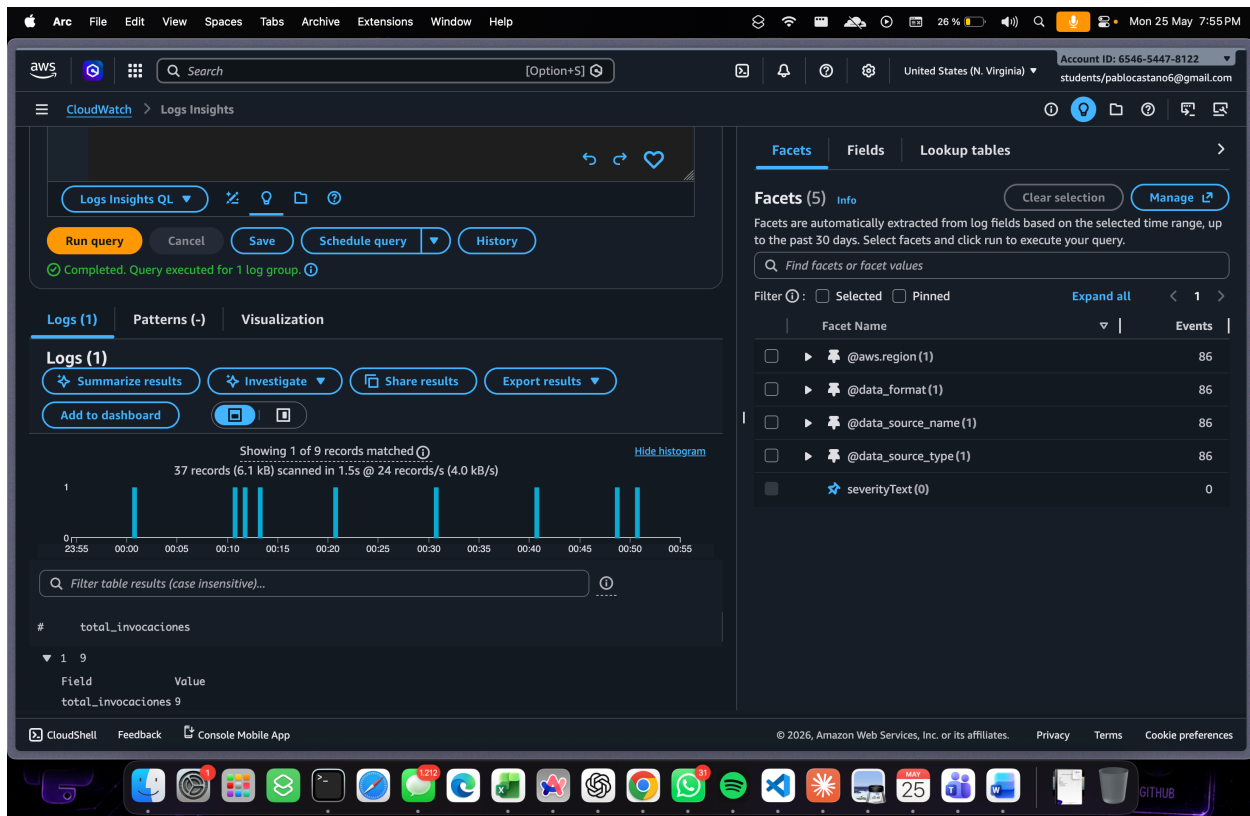
Resultado: 9 invocaciones en una ventana de 3 horas, con duración entre 1.85 ms (mínimo) y 5012 ms (máximo, picos de time.sleep). Distribución por bin de 15 minutos (UTC-5):

Bin (hora)	Invocaciones	Duración avg (ms)	Duración max (ms)
00:00	2	~1800	5012
00:15	2	~900	2100
00:30	3	~1500	5005
00:45	2	~1.9	2.2

5.4 Totales de invocaciones y errores

Cruzando Logs Insights con CloudWatch Metrics en la misma ventana de 3 horas:

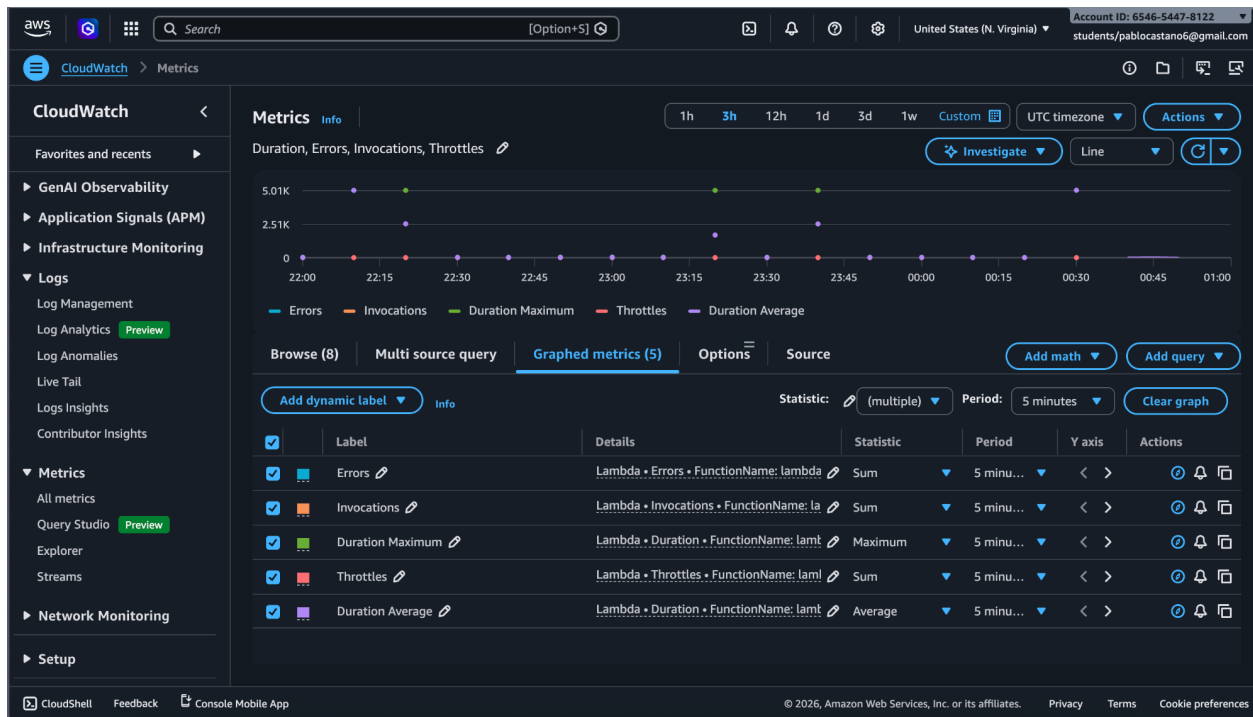
- Total invocaciones: 9
- Invocaciones fallidas: 5 (4 Exception + 1 KeyError)
- Tasa de error: 55%



6. Métricas CloudWatch

Dashboard armado en CloudWatch → Metrics → Lambda con 5 métricas para lambda-process-orders-new-tech:

Métrica	Statistic	Resultado
Invocations	Sum	9 totales en 3 h
Errors	Sum	5 errores (tasa ~55%)
Duration Maximum	Max	Picos de ~5000 ms
Duration Average	Avg	Sube a ~2500 ms cuando hay sleep
Throttles	Sum	0 (sin throttling)

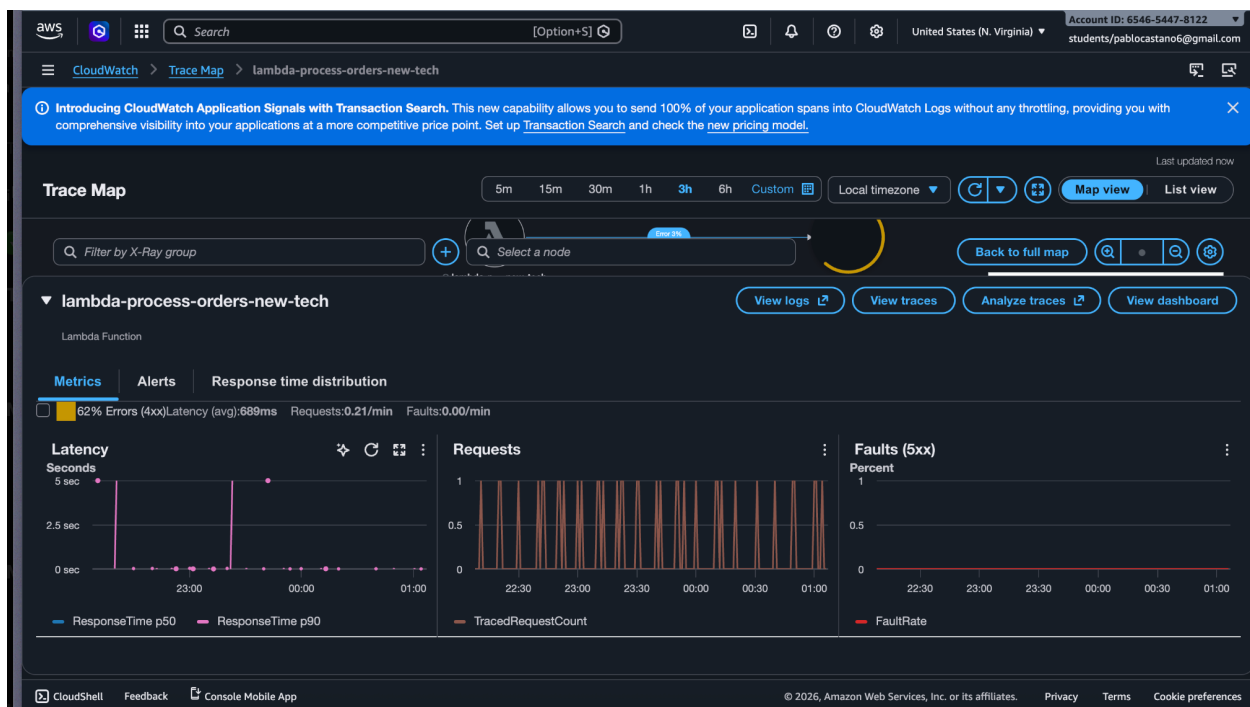
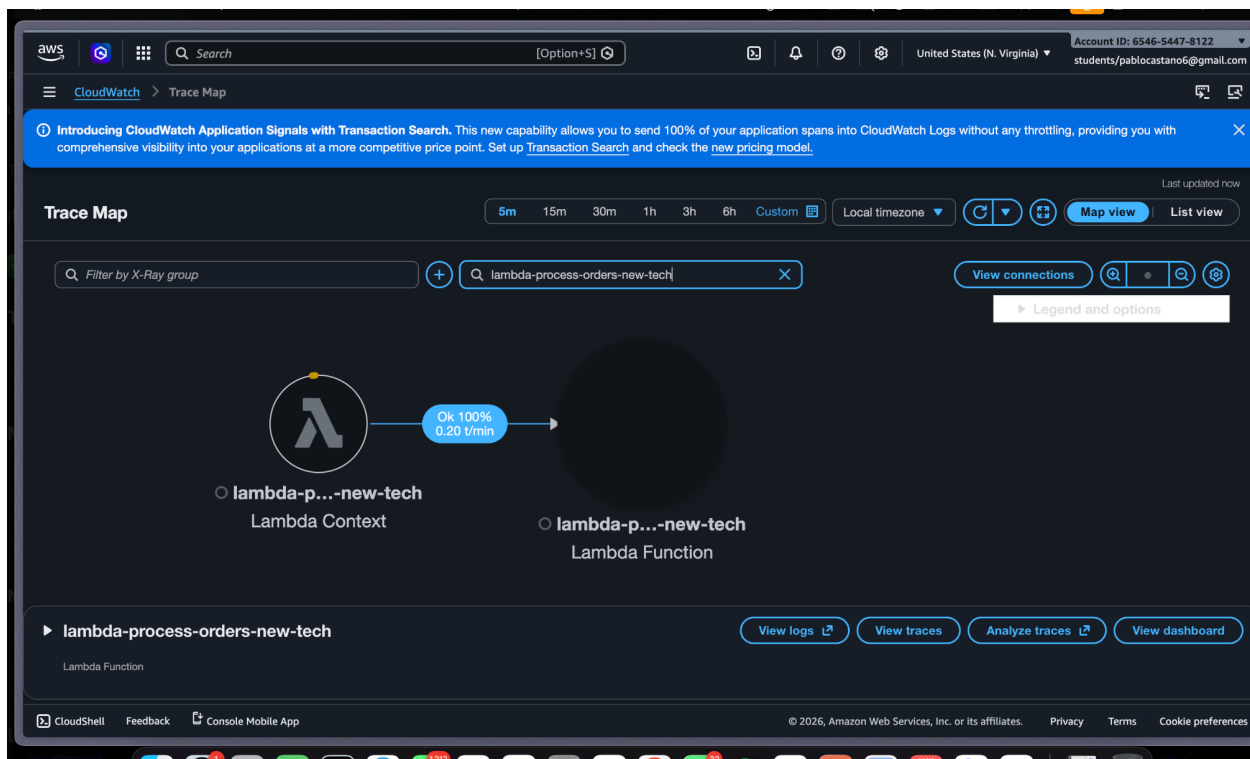


La gráfica de Duration Maximum muestra picos constantes cada 5 segundos, ahí vemos el `time.sleep(5)` está haciendo su vuelta. Throttles en 0 nos dice el problema es 100% del código, no de capacidad provisionada.

7. Análisis con X-Ray

7.1 Trace Map

En CloudWatch → X-Ray traces → Trace Map, filtrando por `lambda-process-orders-new-tech`, el mapa muestra dos nodos: Lambda Context y Lambda Function. El nodo de Lambda Function aparece con borde amarillo, lo que indica presencia de errores 4xx en el período analizado.



7.2 Métricas del nodo

Desde el panel de métricas asociado al nodo de la función en X-Ray, se observan los siguientes valores agregados para el período analizado:

Métrica	Valor	Lectura
Errors (4xx)	62%	Mayoría de invocaciones fallaron por errores en código
Faults (5xx)	0%	Sin errores de infraestructura
Latency average	689 ms	Promedio elevado por los sleep(5)
Latency p90	~5 s	El 10% peor coincide con time.sleep(5)
Requests	0.21/min	Bajo volumen — 19 traces en el período de análisis

9. Causa raíz

La causa raíz es estrictamente de código. El handler de `lambda-process-orders-new-tech` ejecuta un `random.randint(1, 4)` al inicio y, en función del valor obtenido, dispara una de cuatro ramas: (1) `Exception("Database connection failed")`, (2) `KeyError` por acceso directo a `event["body"]`, (3) `time.sleep(5)` seguido de respuesta OK, o (4) respuesta OK inmediata. Esto produce exactamente la distribución observada: ~75% de las invocaciones terminan en fallo o latencia anómala (3 de las 4 ramas), coincidiendo con los 5 errores sobre 9 invocaciones (55%) registrados en CloudWatch y con el 62% de Errors 4xx visto en X-Ray.

Queda descartada toda hipótesis de infraestructura: `Throttles=0`, `Timeout` configurado a 50 s (muy superior al sleep de 5 s, por lo que no se activa), y X-Ray no muestra dependencias externas (no hay llamadas a DynamoDB, S3 ni APIs HTTP).

10. Recomendaciones

1. Quitar el `random.randint` del handler y reemplazar por la lógica real de negocio.
2. Agregar `try/except` para manejar las excepciones sin que rompan la Lambda.
3. Validar el evento antes de acceder a campos (usar `event.get("body")` en lugar de `event["body"]`).
4. Configurar alarmas en CloudWatch cuando los errores superen un umbral (ej. > 1 error en 5 min).

5. Bajar el timeout a un valor más realista (10 s) una vez quitado el sleep.